Slip 1: Q1:

```
import numpy as np
from math import*
def f(x):
    return x**3-5*x-9
def regula_falsi(a,b,tol=1e-6,max_iter=15):
    if f(a)*f(b)>=0:
        print("Error:f(a)&f(b)must have opposite sign")
        return None
    print(f"\n{'Iter':<5} {'a':<12} {'b':<12} {'c':<12} {'f(c)':<12}")
    print("-"*55)
    for i in range(1,max_iter+1):
        c=b-(f(b)-(b-a))/(f(b)-f(a))
        fc=f(c)
        print(f"{i:<5} {a:<12.6f} {b:<12.6f} {c:<12.6f} {c:<12.6f} {fc:25.6f}")
        if abs (fc)<tol:
            return c
        if f(a)*fc<0:
            b=c
        else:
            a=c
    return c
```

o/p:

```
a=2
b=3
root=regula_falsi(a,b)
if root is not None:
    print("approximate root=",round(root,7))
    print("f(root)=",f(root))
```

Q2:1

```
import math
def trapezoidal_rule(func,a,b,n):
    h=(b-a)/n
    integral_sum=0.5*(func(a)+func(b))
    for i in range(1,n):
        integral_sum+=func(a+i*h)
    return integral_sum*h
lower_limit=-3
upper_limit=3
segments=100
result=trapezoidal_rule(lambda x:x*x,lower_limit,upper_limit,segments)
print(f"Approximate integral:{result}")
print(f"Actual answer:2.0")
print(f"Absolute Error:{abs(2.0-result)}")
```

2)

```
def euler_method(f, x0, y0, x_end, h):
    x = x0
    y = y0
    print(f"Step 0: x = {x:.4f}, y = {y:.4f}")
    steps = int((x_end - x0) / h)
    for i in range(1, steps + 1):
        y = y + h * f(x, y)
        x = x + h
        print(f"Step {i}: x = {x:.4f}, y = {y:.4f}")
    return y
if __name__ == "__main__":
    def f(x, y):
        return -x*y**2
```

```
initial_x = 0.0
initial_y = 1.0
target_x = 0.5
step_size = 0.1
print(f"Approximating solution for dy/dx = x + y at x = {target_x}:")
final_y = euler_method(f, initial_x, initial_y, target_x, step_size)
print(f"\nApproximate solution at x = {target_x} is y = {final_y:.4f}")
```

slip 2:

Q1:

```
import numpy as np
from math import*
def f(x):
    return x**3-10*x**2+5
def regula_falsi(a,b,tol=1e-6,max_iter=25):
    if f(a)*f(b)>=0:
        print("Error:f(a)&f(b)must have opposite sign")
        return None
    print(f"\n{'Iter':<5} {'a':<12} {'b':<12} {'c':<12} {'f(c)':<12}")
    print("-"*55)
    for i in range(1,max_iter+1):
        c=b-(f(b)-(b-a))/(f(b)-f(a))
        fc=f(c)
        print(f"{i:<5} {a:<12.6f} {b:<12.6f} {c:<12.6f} {c:<12.6f} {fc:25.6f}")
        if abs (fc)<tol:
            return c
        if f(a)*fc<0:
            b=c
        else:
```

```
        a=c
    return c
```

Q2:

1:

```
 def f(x):
    return 1/x
def simpsons_one_third(a,b,n):
    if n%2!=0:
        raise valueError("the number of intervals (n) must be an even number.")
    h=(b-a)/n
    integral=f(a)+f(b)
    for i in range(1,n):
        x=a+i*h
        if i%2==0:
            integral+=2*f(x)
        else:
            integral+=4*f(x)
    integral*=h/3
    return integral
lower_limit=1
upper_limit=7
intervals=12
result=simpsons_one_third(lower_limit,upper_limit,intervals)
print(f"approximated integral:{result:.4f}")
```

2)

```
import pandas as pd
def generate_divided_difference_table(x, y):
    n=len(x)
    table=[[0.0 for _ in range (n)]for _ in range (n)]
```

```
    for i in range (n):
        table[i][0]=float (y[i])
    for j in range(1, n):
        for i in range (n - j):
            numerator = table[i+1][j-j+(j-1)]-table[i][j-1]
            denominator = x[i+j]-x[i]
            table[i][j] = numerator / denominator
    headers = ["x","f[x]"]+[f"order {j} diff"for j in range(1,n)]
    formatted_data=[]
    for i in range(n):
        row = [x[i]]+[table[i][j] for j in range(n)]
        formatted_data.append(row)
    df = pd.DataFrame(formatted_data, columns=headers)
    return df
x_data =[1,2,5,7]
y_data = [6,9,30,54]
diff_table = generate_divided_difference_table(x_data,y_data)
print("\n=== newton divide difference table ===")
print(diff_table.to_string(index=False))
```

slip 3:

Q1:

```
import numpy as np
import matplotlib.pyplot as plt
def f(x):
    return 3*x-np.cos(x)-1
def bisection(a,b,tol=1e-6,max_iter=20):
    if f(a)*f(b)>=0:
        print('bisection method cannot be applied')
        return None
    for i in range(max_iter+1):
```

```
        c=(a+b)/2
        print(f"{i+1}\t {a:.6f}\t {b:.6f}\t {c:.6f}\t {f(c):.6f}")
        if abs(f(c))<tol:
            print("root found after",i+1,"iteration")
            return c
        if f(a)*f(c)<0:
            b=c
        else:
            a=c
    return (a+b)/2
root=bisection(1,2)
print("approximate root=",root)
x=np.linspace(-2,2,400)
y=f(x)
plt.figure(figsize=(8,5))
plt.plot(x,y,label='y=2*cos(x)-1')
plt.axhline(0,color='black',lw=1)
if root is not None:
    plt.scatter(root,f(root),color='red',label=f'root={root:.6f}')
plt.grid(True)
plt.xlabel("x")
plt.ylabel("y")
plt.title("bisection method root plot")
plt.legend()
plt.show()
```

Q2:

1:

```
import math
def trapezoidal_rule(func,a,b,n):
    h=(b-a)/n
    integral_sum=0.5*(func(a)+func(b))
```

```
    for i in range(1,n):
        integral_sum+=func(a+i*h)
    return integral_sum*h
lower_limit=0
upper_limit=math.pi
segments=100
result=trapezoidal_rule(math.sin,lower_limit,upper_limit,segments)
print(f"Approximate integral:{result}")
print(f"Actual answer:2.0")
print(f"Absolute Error:{abs(2.0-result)}")
```

2:

```
import numpy as np
def forward_difference_table(x, y):
    n = len(x)
    table = np.zeros((n, n))
    for i in range(n):
        table[i][0] = y[i]
    for j in range(1, n):
        for i in range(n - j):
            table[i][j] = table[i + 1][j - 1] - table[i][j - 1]
    return table, n
x = [1,2,3,4,5]
y = [40,60,65,50,18]
table, n = forward_difference_table(x, y)
print("x\tf(x)\t" + "\t".join([f"Δ^{i}y" for i in range(1, n)]))
print("-" * 60)
for i in range(n):
    print(f"{x[i]:<5}", end="")
    for j in range(n - i):
        print(f"\t{table[i][j]:.2f}", end="")
```

```
    print()
```

Slip4:

Q:1

```
import numpy as np
from math import*
def f(x):
    return x**3-2*x+1
def df(x):
    return 3*x**2-2
def newton_raphson(x0,tol=1e-6,max_iter=15):
    print(f"\n{'Iter':<5} {'a':<12} {'b':<12} {'c':<12} {'f(c)':<12}")
    print("-"*55)
    x=x0
    for i in range(1,max_iter+1):
        fx=f(x)
        dfx=df(x)
        if dfx==0:
            print("Error:Derivative is zero.method fails")
            return None
        c=x-fx/dfx
        print(f"{i:<5} {x:<12.6f} {f(x):<15.8f} {f(x):<15.8f} {dfx:<15.8f}")
        if abs (c-x)<tol:
            return c
    return c
```

o/p:

```
x0=2
root=newton_raphson(x0)
if root is not None:
    print("approximate root=",round(root,7))
```

```
    print("f(root)=",f(root))
```

Q2:

1 :

```
import math
def f(x):
    return x*math.exp(x)
def simpsons_one_third(a,b,n):
    if n%2!=0:
        raise valueError("the number of intervals (n) must be an even number.")
    h=(b-a)/n
    integral=f(a)+f(b)
    for i in range(1,n):
        x=a+i*h
        if i%2==0:
            integral+=2*f(x)
        else:
            integral+=4*f(x)
    integral*=h/3
    return integral
lower_limit=1
upper_limit=7
intervals=12
result=simpsons_one_third(lower_limit,upper_limit,intervals)
print(f"approximated integral:{result:.4f}")
```

2:

```
def euler_method(f, x0, y0, x_end, h):
    x = x0
    y = y0
```

```
    print(f"Step 0: x = {x:.4f}, y = {y:.4f}")
    steps = int((x_end - x0) / h)
    for i in range(1, steps + 1):
        y = y + h * f(x, y)
        x = x + h
        print(f"Step {i}: x = {x:.4f}, y = {y:.4f}")
    return y
if __name__ == "__main__":
    def f(x, y):
        return -2*x*y**2
    initial_x = 0.0
    initial_y = 1.0
    target_x = 0.5
    step_size = 0.2
    print(f"Approximating solution for dy/dx = x + y at x = {target_x}:")
    final_y = euler_method(f, initial_x, initial_y, target_x, step_size)
    print(f"\nApproximate solution at x = {target_x} is y = {final_y:.4f}")
```

Slip 5:

1:

```
import matplotlib.pyplot as plt
import numpy as np
from pylab import *
from math import *
def f(x):
    return x*np.log(x)
def bisection(a, b, tol=1e-4, max_iter=25):
    if f(a) * f(b) >= 0:
        print("Error: f(a) & f(b) must have opposite sign")
        return None
    print(f"{'Iter':<5}{'a':<12}{'b':<12}{'c':<12}{'f(c)':<12}")
```

```
    print("-" * 55)
    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        fc = f(c)
        print(f"{i:<5}{a:<12.6f}{b:<12.6f}{c:<12.6f}{fc:<12.6f}")
        if abs(fc) < tol:
            return c
        if f(a) * fc < 0:
            b = c
        else:
            a = c
    return (a + b) / 2
root = bisection(0.5,2)
print("approximate root", root)
x = np.linspace(0.5,24,500)
y = f(x)
plt.title('graph of Bisection')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.plot(x, y, label='f(x)=x*log(x)', color='blue')
plt.legend(loc=1)
plt.show()
```

Q:2:

1:

```
import math
def trapezoidal_rule(func, a, b, n):
    h = (b - a) / n
    integral_sum = 0.5 * (func(a) + func(b))
    for i in range(1, n):
        integral_sum += func(a + i * h)
    return integral_sum * h
```

```
lower_limit = 2
upper_limit = 5
segments = 100
result = trapezoidal_rule(lambda x:x*x-2*x+1, lower_limit, upper_limit, segments)
print(f"Approximated Integral: {result}")
print(f"Actual Answer: 2.0")
print(f"Absolute Error: {abs(2.0 - result)}")
```

2:

```
def f(x, y):
    return 1+y**2

x = 0.0
y = 2.0
h = 0.1

for i in range(2):
    k1 = h * f(x, y)
    k2 = h * f(x + h, y + k1)

    y = y + (k1 + k2) / 2
    x = x + h

    print("y(", round(x, 1), ") =", round(y, 4))
```

Slip6:

Q1:

```
import numpy as np
from math import*
def f(x):
    return x**3-x**2-2
def df(x):
    return 3*x**2-2*x
```

```
def newton_raphson(x0,tol=1e-6,max_iter=10):
    print(f"{"Iter":<5}{"x":<15}{"f(x)":<15}{"f\n(x)":<15}" )
    print("-"*55)
    x=x0
    for i in range(1,max_iter+1):
        fx=f(x)
        dfx=df(x)
        if dfx==0:
            print("Error:Derivative is zero.method fails.")
            return None
        c=x-fx/dfx
        print(f"{i:<5}{x:<15.8f}{f(x):<15.8f}{dfx:<15.8f}")
        if abs(c-x)<tol:
             return c
    return x
x0=5
root=newton_raphson(x0)
if root is not None:
    print("Approximate Root=",round(root,7))
    print("f(root)=",f(root))
```

Q2:

1:

```
import math
def f(x):
    return x**2-10
def simpsons_one_third(a,b,n):
    if n%2!=0:
        raise valueError("the number of intervals (n) must be an even number.")
    h=(b-a)/n
    integral=f(a)+f(b)
    for i in range(1,n):
```

```
        x=a+i*h
        if i%2==0:
            integral+=2*f(x)
        else:
            integral+=4*f(x)
    integral*=h/3
    return integral
lower_limit=1
upper_limit=2
intervals=12
result=simpsons_one_third(lower_limit,upper_limit,intervals)
print(f"approximated integral:{result:.4f}")
```

2:

```
def f(x, y):
    return y**2 + x**2

x = 1.0
y = 0.0
h = 0.1
xn = 1.4

while x < xn:
    k1 = h * f(x, y)
    k2 = h * f(x + h/2, y + k1/2)
    k3 = h * f(x + h/2, y + k2/2)
    k4 = h * f(x + h, y + k3)

    y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
    x = round(x + h, 10)
```

```
    print("x =", round(x, 1), "y =", round(y, 6))


print("The value of y(1.4) =", round(y, 6))
```

Slip 7:

Q:1

```
import numpy as np
from math import*
def f(x):
    return x**3-9*x+1
def regula_falsi(a,b,tol=1e-6,max_iter=15):
    if f(a)*f(b)>=0:
        print("Error:f(a)&f(b)must have opposite sign")
        return None
    print(f"\n{'Iter':<5} {'a':<12} {'b':<12} {'c':<12} {'f(c)':<12}")
    print("-"*55)
    for i in range(1,max_iter+1):
        c=b-(f(b)-(b-a))/(f(b)-f(a))
        fc=f(c)
        print(f"{i:<5} {a:<12.6f} {b:<12.6f} {c:<12.6f} {c:<12.6f} {fc:25.6f}")
        if abs (fc)<tol:
            return c
        if f(a)*fc<0:
            b=c
        else:
            a=c
    return c
```

Q2:

1:

```
def f(x):
```

```
    return 1+x**3
def simpson_3_8(a, b, n):
    if n % 3 != 0:
        raise ValueError("The number of intervals (n) must be a multiple of 3.")
    h = (b - a) / n
    integral_sum = f(a) + f(b)
    for i in range(1, n):
        if i % 3 == 0:
            integral_sum += 2 * f(a + i * h)
        else:
            integral_sum += 3 * f(a + i * h)
    return (3 * h / 8) * integral_sum
lower_limit = 0
upper_limit = 5
intervals=12
result=simpsons_3_8(lower_limit,upper_limit,intervals)
print(f"approximated integral:{result:.4f}")
```

2:

```
def euler_method(f, x0, y0, x_end, h):
    x = x0
    y = y0
    print(f"Step 0: x = {x:.4f}, y = {y:.4f}")
    steps = int((x_end - x0) / h)
    for i in range(1, steps + 1):
        y = y + h * f(x, y)
        x = x + h
        print(f"Step {i}: x = {x:.4f}, y = {y:.4f}")
    return y
```

```
if __name__ == "__main__":
    def f(x, y):
        return 2-x*y**2
    initial_x = 0.0
    initial_y = 1.0
    target_x = 0.5
    step_size = 0.2
    print(f"Approximating solution for dy/dx = x + y at x = {target_x}:")
    final_y = euler_method(f, initial_x, initial_y, target_x, step_size)
    print(f"\nApproximate solution at x = {target_x} is y = {final_y:.4f}")
```

SLIP 8:

Q:1

```
import matplotlib.pyplot as plt
import numpy as np
from pylab import *
from math import *
def f(x):
    return x-np.cos(x)
def bisection(a, b, tol=1e-6, max_iter=100):
    if f(a) * f(b) >= 0:
        print("Error: f(a) & f(b) must have opposite sign")
        return None
    print(f"{'Iter':<5}{'a':<12}{'b':<12}{'c':<12}{'f(c)':<12}")
    print("-" * 55)
    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        fc = f(c)
        print(f"{i:<5}{a:<12.6f}{b:<12.6f}{c:<12.6f}{fc:<12.6f}")
        if abs(fc) < tol:
```

```
            return c
        if f(a) * fc < 0:
            b = c
        else:
            a = c
    return (a + b) / 2
root = bisection(1, 2)
print("approximate root", root)
x = np.linspace(-4, 4, 500)
y = f(x)
plt.title('graph of Bisection')
plt.xlabel('x-axis')
```

Q:2

1:

```
def f(x, y):
    return y-x


x = 0.0
y = 2.0
h = 0.1


for i in range(2):
    k1 = h * f(x, y)
    k2 = h * f(x + h, y + k1)


    y = y + (k1 + k2) / 2
    x = x + h


    print("y(", round(x, 1), ") =", round(y, 4))
```

2:

```
import math
def trapezoidal_rule(func, a, b, n):
    h = (b - a) / n
    integral_sum = 0.5 * (func(a) + func(b))
    for i in range(1, n):
        integral_sum += func(a + i * h)
    return integral_sum * h
lower_limit = 2
upper_limit = 5
segments = 100
result = trapezoidal_rule(lambda x:4*x-3*x**2, lower_limit, upper_limit, segments)
print(f"Approximated Integral: {result}")
print(f"Actual Answer: 2.0")
print(f"Absolute Error: {abs(2.0 - result)}")
```

SLIP 9 :

Q1:

```
import numpy as np
from math import*
def f(x):
    return x**3-4*x+1
def regula_falsi(a,b,tol=1e-5,max_iter=25):
    if f(a)*f(b)>=0:
        print("Error:f(a)&f(b)must have opposite sign")
        return None
    print(f"\n{'Iter':<5} {'a':<12} {'b':<12} {'c':<12} {'f(c)':<12}")
    print("-"*55)
    for i in range(1,max_iter+1):
        c=b-(f(b)-(b-a))/(f(b)-f(a))
        fc=f(c)
        print(f"{i:<5} {a:<12.6f} {b:<12.6f} {c:<12.6f} {c:<12.6f} {fc:25.6f}")
```

```
        if abs (fc)<tol:
            return c
        if f(a)*fc<0:
            b=c
        else:
            a=c
    return c
```

Q2:

1:

```
def euler_method(f, x0, y0, x_end, h):
    x = x0
    y = y0
    print(f"Step 0: x = {x:.4f}, y = {y:.4f}")
    steps = int((x_end - x0) / h)
    for i in range(1, steps + 1):
        y = y + h * f(x, y)
        x = x + h
        print(f"Step {i}: x = {x:.4f}, y = {y:.4f}")
    return y
if __name__ == "__main__":
    def f(x, y):
        return x**2+y
    initial_x = 0.0
    initial_y = 1.0
    target_x = 0.5
    step_size = 0.05
    print(f"Approximating solution for dy/dx = x + y at x = {target_x}:")
    final_y = euler_method(f, initial_x, initial_y, target_x, step_size)
    print(f"\nApproximate solution at x = {target_x} is y = {final_y:.4f}")
```

2:

```
import math
def f(x):
    return 1/1+x
def simpsons_one_third(a,b,n):
    if n%2!=0:
        raise valueError("the number of intervals (n) must be an even number.")
    h=(b-a)/n
    integral=f(a)+f(b)
    for i in range(1,n):
        x=a+i*h
        if i%2==0:
            integral+=2*f(x)
        else:
            integral+=4*f(x)
    integral*=h/3
    return integral
lower_limit=0
upper_limit=1
intervals=8
result=simpsons_one_third(lower_limit,upper_limit,intervals)
print(f"approximated integral:{result:.4f}")
```

SLIP 10:

Q1:

```
import numpy as np
from math import*
def f(x):
    return 2*x**3-3*x-6
def df(x):
    return 6*x**2-3
def newton_raphson(x0,tol=1e-6,max_iter=10):
```

```
    print(f"{"Iter":<5}{"x":<15}{"f(x)":<15}{"f\n(x)":<15}" )
    print("-"*55)
    x=x0
    for i in range(1,max_iter+1):
        fx=f(x)
        dfx=df(x)
        if dfx==0:
            print("Error:Derivative is zero.method fails.")
            return None
        c=x-fx/dfx
        print(f"{i:<5}{x:<15.8f}{f(x):<15.8f}{dfx:<15.8f}")
        if abs(c-x)<tol:
            return c
    return x
x0=5
root=newton_raphson(x0)
if root is not None:
    print("Approximate Root=",round(root,7))
    print("f(root)=",f(root))
```

Q2:

1:

```
def euler_method(f, x0, y0, x_end, h):
    x = x0
    y = y0
    print(f"Step 0: x = {x:.4f}, y = {y:.4f}")
    steps = int((x_end - x0) / h)
    for i in range(1, steps + 1):
        y = y + h * f(x, y)
        x = x + h
        print(f"Step {i}: x = {x:.4f}, y = {y:.4f}")
    return y
```

```
if __name__ == "__main__":
    def f(x, y):
        return x*y
    initial_x = 0.0
    initial_y = 1.0
    target_x = 0.5
    step_size = 0.1
    print(f"Approximating solution for dy/dx = x + y at x = {target_x}:")
    final_y = euler_method(f, initial_x, initial_y, target_x, step_size)
    print(f"\nApproximate solution at x = {target_x} is y = {final_y:.4f}")
```

2:

```
def f(x):
    return 1/1+x**4
def simpson_3_8(a, b, n):
    if n % 3 != 0:
        raise ValueError("The number of intervals (n) must be a multiple of 3.")
    h = (b - a) / n
    integral_sum = f(a) + f(b)
    for i in range(1, n):
        if i % 3 == 0:
            integral_sum += 2 * f(a + i * h)
        else:
            integral_sum += 3 * f(a + i * h)
    return (3 * h / 8) * integral_sum
lower_limit = 0
upper_limit = 6
intervals=12
result=simpsons_3_8(lower_limit,upper_limit,intervals)
print(f"approximated integral:{result:.4f}")
```